

Building Docker Images with a Dockerfile

Virtualizzazione e integrazione di sistemi

Vittorio Ghini

a.a. 2024-25

Ingegneria e scienze informatiche

Università di Bologna

Contents

- Operational aspects
 - Dockerfile and Dockerfile commands
 - Image name
 - Build and Run example
 - Inspecting image
- Theoretical aspects
 - Filesystem of running container
 - Layered structure of container image filesystem
 - Incremental, shared,
 - Build cache
 - Storage drivers
 - overlay2 (default), aufs, btrfs

1. Dockerfile

- A **Dockerfile** is a script that contains commands and instructions that will be automatically executed in sequence in the docker environment for building a new docker image.
- A dockerfile contains a collection of dockerfile commands and operating system commands (ex: Linux commands).
- Basic **dockerfile commands** are:
 - **FROM**
 - **RUN**
 - **ADD**
 - **COPY**
 - **ENV**
 - **ENTRYPOINT**
 - **CMD**
 - **WORKDIR**
 - **USER**
 - **VOLUME**
- Other **dockerfile commands** are:
 - **ARG**
 - **EXPOSE**
 - **HEALTHCHECK**
 - **LABEL**
 - **ONBUILD**
 - **SHELL**
 - **STOPSIGNAL**

1.1. Dockerfile command

(1/3)

For details, referes to <https://docs.docker.com/reference/dockerfile/>

- **FROM**

The base image for building a new image. This command must be on top of the dockerfile.

- **RUN**

Used to execute a command on the container during the build process of the docker image.

- **COPY**

Copy a file from the host machine to the new docker image.

- **ADD**

Copy a file from the host machine to the new docker image. Same as COPY but there is an option to use an URL for the source file to be copied, docker will then download that file to the destination directory.

- **ENV**

Define an environment variable. After a variable has been define it can also be use in the other commands and persists in the running containers.

1.1. Dockerfile command

(2/3)

- **ENTRYPOINT**

Define the **default and not replaceable command** that will be executed when the container is running, despite the "docker run" command specifying a different command.

If there is no ENTRYPOINT, the one from the starting image is placed in the image.

- **CMD**

Used for executing a **replaceable command** when we build a new container from the docker image. if there is no CMD, the one from the starting image is places in the image.

This command will be replaced by the one specified by docker run.

- **WORKDIR**

This is directive for CMD command to be executed.

- **USER**

Set the user or UID for the container created with the image.

- **VOLUME**

Enable access/linked directory between the container and the host machine.

1.1. Dockerfile command

(3/3)

- **ARG**
Use build-time variables.
- **EXPOSE**
Describe which ports your application is listening on.
- **HEALTHCHECK**
Check a container's health on startup.
- **LABEL**
Add metadata to an image.

Less frequently used commands

- **ONBUILD**
Specify instructions for when the image will be used in a build.
- **SHELL.**
Set the default shell of an image (shell used in bash form of both entrypoint and cmd commands).
- **STOPSIGNAL**
Specify the system call signal for exiting a container

1.2. Ispezione di una immagine (1/2)

- Il comando: **docker image inspect NOME_IMMAGINE** consente di vedere alcune informazioni sull'immagine contenuta nel registry locale, ad esempio i valori di Entrypoint, Cmd, User, Workingdir, Volume, Architecture, OS, e l'Id, oltre che agli identificatory dei layers.
- Ad esempio, il comando: **docker image inspect ubuntu** visualizza il seguente output (tagliato) che normalmente è in formato Json:

```
[ { "Id": "sha256:ca2b0f26964cf2e80ba3e084d5983dab293fdb87485dc6445f3f7bbfc89d7459",  
  "DockerVersion": "24.0.5",  
  "Config": {  
    "Hostname": "",  
    "User": "",  
    "Env": [ "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin" ],  
    "Cmd": [ "/bin/bash" ],  
    "Volumes": null,  
    "WorkingDir": "",  
    "Entrypoint": null,  
  },  
  "Architecture": "amd64",  
  "Os": "linux",  
  "GraphDriver": { "Name": "overlay2" },  
  "RootFS": {  
    "Type": "layers",  
    "Layers": [ "sha256:5498e8c22f6996f25ef193ee58617d5b37e2a96decf22e72de13c3b34e147591" ]  
  },  
}
```

1.2. Ispezione di una immagine (2/2)

- L'output del comando `inspect` normalmente è in formato Json.
- Il comando `inspect` consente di specificare una opzione **`--format`** con cui è possibile selezionare qualche specifico campo di quell'output da visualizzare.
- L'opzione `--format` vuole come argomento una selezione Json dell'output, che è composta dalla sequenza di campi, preceduti da un `.` puntino, che bisogna attraversare per arrivare a quello di interesse, messi tra doppie parentesi graffe aperte e chiuse.

Ad esempio, per selezionare l'identificatore `Id` dell'immagine posso usare:

```
docker image inspect --format='{{.Id}}' NOME_IMMAGINE
```

ottenendo in output

```
sha256:ca2b0f26964cf2e80ba3e084d5983dab293fdb87485dc6445f3f7bbfc89d7459
```

Ad esempio, per selezionare tutti i layers dell'immagine, posso usare:

```
docker image inspect --format='{{.RootFS.Layers}}' NOME_IMMAGINE
```

ottenendo in output

```
[sha256:5498e8c22f6996f25ef193ee58617d5b37e2a96decf22e72de13c3b34e147591,  
sha256:7630e8c22f6996f25ef193ee58617d5b37e2a96decf22e72au54d6f23f676742 ]
```

che ci indica l'esistenza di due layers.

1.2. Ispezione immagini (2/3)

- **Esercizi : strati di immagine derivata**
- Scaricare l'immagine di ubuntu
- Visualizzare il comando principale ENTRYPOINT e CMD con inspect
- Scaricare l'immagine di mysql
- Visualizzare il comando principale ENTRYPOINT e CMD con inspect
- Visualizzare ENV e PATH
- Scaricare l'immagine di adminer
- Visualizzare il comando principale ENTRYPOINT e CMD con inspect
- Visualizzare ENV, layers, ...

1.3. ENTRYPOINT vs CMD (1/2)

- Sia ENTRYPOINT che CMD concorrono a definire il comando principale che viene eseguito appena il container è running (Up).
- Se ci sono sia ENTRYPOINT che CMD senza che docker run specifichi un comando allora il comando è dato da ENTRYPOINT seguito da CMD
- La differenza tra CMD e ENTRYPOINT è che **eventuali** “comando e suoi argomenti” specificati dal comando docker run
 - **sostituiscono** il comando specificato da CMD nel Dockerfile,
 - **si aggiungono** come ulteriori argomenti al comando specificato da ENTRYPOINT.
- Quindi, il comando principale eseguito dal container dipenderà dall’esistenza o non esistenza dei 3 comandi specificati da ENTRYPOINT, CMD e da docker run, secondo la seguente tabella:

ENTRYPOINT	CMD	Docker run optional cmdline	Result: Main command
Entry_line			Entry_line
Entry_line	Cmd_line (*)		Entry_line Cmd_line
Entry_line		Opt_cmdline	Entry_line Opt_cmdline
Entry_line	Cmd_line	Opt_cmdline	Entry_line Opt_cmdline
	Cmd_line		Cmd_line
		Opt_cmdline	Opt_cmdline
	Cmd_line	Opt_cmdline	Opt_cmdline

(*) caso particolare
Vedi tra
due slide

1.3. ENTRYPOINT vs CMD (2/2)

- Esempio:

```
ENTRYPOINT [ "/bin/bash", "-c" ]
```

```
CMD [ "./hello.sh", "alfa", "beta", "gamma" ]
```

- Se eseguo 'docker run myimages' senza specificare un comando, allora verrà eseguito nel container la seguente riga di comando, data dalla concatenazione di ENTRYPOINT e CMD:

```
/bin/bash -c ./hello.sh alfa beta gamma
```

- Se eseguo 'docker run myimages /home/vic/script.sh ciao come va', allora verrà eseguito nel container la seguente riga di comando:

```
/bin/bash -c /home/vic/script.sh ciao come va
```

- **Se voglio usare una immagine di container per poter lanciare un comando qualsiasi, NON devo mettere ENTRYPOINT ma devo mettere SOLO il CMD, che potrà essere sostituito completamente da un eventuale comando che sarà specificato in docker run.**

- Ad esempio, se nel Dockerfile ho messo solo il CMD e poi ho costruito l'immagine

```
CMD [ "./hello.sh", "alfa", "beta", "gamma" ]
```

- Se eseguo 'docker run myimages /home/vic/script.sh ciao come va', allora verrà eseguito nel container la seguente riga di comando:

```
/home/vic/script.sh ciao come va
```

- perché il CMD è stato completamente sostituito

1.4. Simple Dockerfile Example (1/4)

In host directory ./ FILE **hello.sh**

```
#!/bin/bash
```

```
echo inizio
```

```
./hello1.sh
```

In host directory ./ FILE **hello1.sh**

```
#!/bin/bash
```

```
echo vaf | nc www.cs.unibo.it 80
```

File permissions are **666**

1.4. Simple Dockerfile Example (2/4)

In host directory ./ FILE **Dockerfile**

```
FROM ubuntu:latest
```

```
MAINTAINER vic ovvero io
```

```
ENV MYHOME=/home/
```

```
RUN apt-get update
```

```
RUN apt-get -y install net-tools
```

```
RUN apt-get -y install netcat-openbsd
```

```
ADD hello.sh /home/hello.sh
```

```
ADD hello1.sh /home/hello1.sh
```

```
RUN chmod 777 ${MYHOME}/hello.sh
```

```
RUN chmod 777 ${MYHOME}/hello1.sh
```

```
WORKDIR /home
```

```
ENTRYPOINT [ "./hello.sh" ]
```

1.4. Simple Dockerfile Example (3/4)

BUILD

```
docker build -t "simple_netcat:1.0" .
```

VERIFY

```
docker images "simple_netcat:1.0"
```

RUN

```
docker run simple_netcat:1.0
```

OUTPUT

see next slide

1.4. Simple Dockerfile Example (4/4)

OUTPUT

inizio

HTTP/1.1 400 Bad Request

Date: Wed, 27 Apr 2022 05:25:37 GMT

Server: Apache/2.4.10 (Debian)

Content-Length: 311

Connection: close

Content-Type: text/html; charset=iso-8859-1

```
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
```

```
<html><head>
```

```
<title>400 Bad Request</title>
```

```
</head><body>
```

```
<h1>Bad Request</h1>
```

```
<p>Your browser sent a request that this server could not understand.<br />
```

```
</p>
```

```
<hr>
```

```
<address>Apache/2.4.10 (Debian) Server at serpina.cs.unibo.it Port 80</address>
```

```
</body></html>
```

3.1. Exec form e Shell form (1/3)

- C'è un ulteriore dettaglio che complica la situazione:
- I comandi in ENTRYPOINT e in COMMAND possono essere specificati in due forme:
- **Exec form (o formato json):** ENTRYPOINT ["./program.exe", "uno", "due", "tre"]
 - viene eseguito esattamente il comando: **./program.exe uno due tre**
 - servono i doppi apici " attorno a ciascuna parte.
- **Shell form:** ENTRYPOINT ./hello.sh alfa beta gamma
 - **viene lanciata una shell dentro cui viene eseguito il comando specificato,**
 - in sostanza viene eseguito il comando: **/bin/sh -c ./hello.sh alfa beta gamma**
 - La shell usata di default è /bin/sh ma se nel Dockerfile è presente il comando SHELL si usa la shell specificata dal comando SHELL. E' preferita /bin/sh perché esiste in tutte le immagini busybox
- Il caso particolare, **citato con (*) due slides indietro**, è questo:
Se ENTRYPOINT è nella shell form e CMD è nella exec form, allora il comando specificato da CMD non viene considerato e non viene eseguito.
- Vedere qui per dettagli sui comandi nei Dockerfile:

<https://docs.docker.com/reference/dockerfile/#understand-how-cmd-and-entrypoint-interact>

<https://docs.docker.com/reference/dockerfile/#shell-and-exec-form>

3.2. Exec form - svolgere compiti di Init

- **Usando il formato exec** (detto anche formato json) il **processo indicato viene eseguito con PID 1** e perciò dovrebbe svolgere i compiti assegnati al processo Init (quello che ha PID 1). Tra questi compiti c'è eliminare i processi zombie nel container (child reaping) che potrebbero appesantire il Sistema e reagire a segnali SIGTERM e SIGKILL.
- Se invece **si indica il processo con il formato shell**, allora il **processo viene eseguito in una shell, e sarà questa shell ad avere PID 1**, non il nostro processo. Ma **la shell non si occupa di eliminare gli zombie**.
- Perciò, se nel mio container so che verranno generate degli zombie, se il mio processo deve occuparsi di eliminare gli zombie allora deve avere PID 1 e devo lanciarlo col formato exec.
- Se però il mio processo non è capace di gestire gli zombie, allora **devo mettere come comando principale un sostituto del processo INIT, devo chiamarlo col formato exec** e far sì che questo **sostituto di init poi chiami il mio processo**.
- **Posso usare tini**, è un processo init minimale progettato proprio per i container Docker.
 - È leggero e efficace.
 - Gestisce i processi figli zombie.
 - E' spesso incluso nelle distribuzioni principali
 - Ha come limite che può lanciare solo un comando.
- In alternativa, posso usare **s6-overlay**, soprattutto se devo lanciare in esecuzione più servizi.
- Esiste anche dumb-init ma è poco usato.
- <https://docs.docker.com/reference/build-checks/json-args-recommended/>

3.3. Usare tini come sostituto di Init

- **USARE TINI COME SOSTITUTO DI INIT.**
- **Uso tini come comando principale e metto il mio programma come comando sostituibile**

Dockerfile

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y tini                # Installa tini
ENTRYPOINT ["/usr/bin/tini", "--"]                          # Imposta tini come processo init
CMD ["/path/to/PROC_A"]                                     # Esegui process principale (PROC_A)
```

La coppia di meno -- dice a Tini che quello che segue è il comando che va lanciato

Eseguire come: `docker run MYIMAGE`

- **Uso tini e il mio comando come comando principale**

Dockerfile

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y tini                # Installa tini
ENTRYPOINT [ "/usr/bin/tini", "--", "/path/to/PROC_A" ]      # Imposta tini
```

mio proc

La coppia di meno -- dice a Tini che quello che segue è il comando che va lanciato

Potrei anche usare:

```
ENTRYPOINT ["/usr/bin/tini", "--", "/bin/bash", "-c" ]
```

3.4. Quando usare Shell form

- Nonostante sarebbe meglio usare la exec form, ci sono casi in cui diventa impossibile.
- Occorre usare la shell form se devo specificare un comando composito in cui occorre eseguire espansioni bash oppure in cui ci sono operatori di congiunzione o condizionali, come `| ( && || ; )` che sono interpretate dalla shell.
- E' però facile scrivere un Dockerfile che include uno script inline che viene eseguito nella exec form. Ad esempio:

```
FROM alpine
RUN apk add bash
COPY --chmod=755 <<EOT
/entrypoint.sh
#!/usr/bin/env bash
set -e
my-background-process &
my-program start
EOT
ENTRYPOINT ["/entrypoint.sh"]
```

```
FROM alpine
RUN apk --no-cache bash tini
COPY --chmod=755 <<EOT /entrypoint.sh
#!/usr/bin/env bash
set -e
my-background-process &
my-program start
EOT
ENTRYPOINT ["/usr/bin/tini", "--", "/entrypoint.sh"]
```

3.5. comando SHELL

- La shell di default che viene usata per eseguire i comandi specificati usando il formato shell in ENTRYPOINT e CMD normalmente è: `[ "/bin/sh", "-c" ]`
- Se mi serve modificarla, posso usare il comando shell così:
`SHELL [ "/bin/bash" , "-c" ]`
- Ad esempio, se nel Dockerfile ho questi due comandi
`SHELL [ "/bin/bash" , "-c" ]`
`ENTRYPOINT "echo ciao; cd /home/vic && echo ok"`

Dopo la creazione il container eseguirà:

```
"/bin/bash" "-c" "echo ciao; cd /home/vic && echo ok"
```

3.6. ENV e ARG - confronto (1/3)

- ENV setta variabili di ambiente dentro il container, ma possono essere usate anche nel Dockerfile a build time.
 - Eventuali variabili di ambiente **passate a riga di comando a run-time** (quando creo il container con docker run) mediante **ripetute opzioni --env**, sovrascrivono le inizializzazioni fatte con ENV nel Dockerfile
 - Però devo stare attento a come ho usato quelle ENV nel Dockerfile, potrei avere già assegnato in forma definitiva i valori stabiliti con ENV (vedi esempio prossima slide dove stampo delle ENV in uno script, notare che uso \
-
- ARG definisce variabili passate a riga di comando che sono usabili SOLO nel Dockerfile.
 - Se mi serve usare un valore ARG anche dentro al container, devo assegnarlo ad una variabile di ambiente con ENV
 - Eventuali valori ARG **passati a riga di comando nel build**, mediante le **opzioni --build-arg** sovrascrivono le inizializzazioni fatte con ARG

3.6. ENV e ARG - confronto (2/3)

- esempio di Dockerfile con ARG e ENV

```
FROM ubuntu
ARG ARG1=arg1 ARG2=arg2 ARG3=arg3
ENV ENV1=env1 ENV2=env2 ENV3=${ARG3}                                <-- assegno a ENV valore di ARG

# CREO uno script da eseguire come comando principale
# Notare che per poter sovrascrivere a runtime le variabili ENV nello script devo quotarle con \
COPY --chmod=755 <<EOT /entrypoint.sh
#!/usr/bin/env bash
set -e
echo ARG1 is $ARG1 ARG2 is $ARG2 ARG3 is $ARG3 | tee /mio.txt
echo ENV1 is \ $ENV1 ENV2 is \ $ENV2 ENV3 is \ $ENV3 | tee -a /mio.txt    <-- stampo ENV, see \
EOT

# nella forma EXEC non posso passare variabili, quindi le metto nello script
ENTRYPOINT [ "/bin/bash", "-c", "/entrypoint.sh" ]
```

3.6. ENV e ARG - confronto (3/3)

Un valore stabilito con ARG può essere assegnato a variabile ENV

```
docker build -t argenv .  
docker run --rm -it --name ubu argenv  
ottengo in output:  
ARG1 is arg1 ARG2 is arg2 ARG3 is arg3  
ENV1 is env1 ENV2 is env2 ENV3 is arg3
```

Posso sovrascrivere **a build time** un valore stabilito con ARG e vederlo assegnato a ENV

```
docker build --build-arg ARG3=nuovoarg3 --build-arg ARG2=nuovoarg2 -t argenv .  
docker run --rm -it --name ubu argenv  
ottengo in output:  
ARG1 is arg1 ARG2 is nuovoarg2 ARG3 is nuovoarg3  
ENV1 is env1 ENV2 is env2 ENV3 is nuovoarg3
```

Posso sovrascrivere **a run time** un valore stabilito con ENV anche se dipendeva da un ARG

```
docker build --build-arg ARG3=nuovoarg3 --build-arg ARG2=nuovoarg2 -t argenv .  
docker run --rm -it --name ubu --env ENV3=runtimeenv3 argenv  
ottengo in output:  
ARG1 is arg1 ARG2 is nuovoarg2 ARG3 is nuovoarg3  
ENV1 is env1 ENV2 is env2 ENV3 is runtimeenv3
```

3.7. RUN command details

- Ogni comando RUN che va a modificare il filesystem crea un layer
- Per evitare il sovrapposri di numerosi layers si può concatenare i comandi dentro un singolo comando RUN con degli operatori bash di esecuzione condizionale (&&)

```
RUN apt update && apt install wget gcc curl && apt autoclean
```

- Oppure si può utilizzare un equivalente dell'operatore Here Document <<

```
RUN <<EOF
apt update
apt install wget curl
apt install gcc
apt autoclean
EOF
```


4. Nome di immagine docker (1/2)

- Il nome completo di un'immagine Docker è composta da quattro parti, di cui una sola obbligatoria

`[REGISTRY_HOST]/[NAMESPACE]/REPOSITORY:[TAG]`

- REGISTRY_HOST** → (Facoltativo) Indica l'host del registry. Deve avere almeno un punto, come i nomi di dominio, e può specificare una porta, preceduta da :
Ad esempio `docker.io`, `ghcr.io`, `my-registry.com:5000`
- NAMESPACE** → (Facoltativo) Indica l'utente o l'organizzazione proprietaria dell'immagine.
- REPOSITORY** → Nome logico dell'immagine (es. `ubuntu`, `nginx`).
- TAG** → (Facoltativo) Specifica la versione dell'immagine (es. `:latest`, `:1.0`).

Per le immagini sul registry `docker hub`, se nella richiesta

ometto il namespace viene messo per default **library** viene

ometto il tag viene messo per default **:latest**

Quindi una richiesta `ubuntu` fatta a `docker hub` diventa `docker.io/library/ubuntu:latest`

- Esempi di nomi completi

`my-registry.com:5000/vic/simple_netcat:1.3`

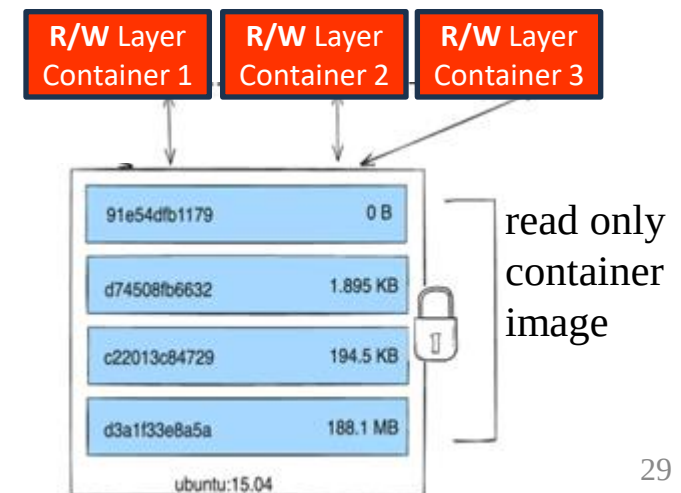
`marcoconti/doubleweb`

4. Nome di immagine docker (2/2)

- Se ho un daemon docker configurato con le impostazioni di default, cioè **che usa docker hub come registry remoto predefinito da cui scaricare le immagini**, allora per riferirmi ad una immagine e caricarla in locale dovrò usare il nome completo o queste abbreviazioni:
- **Immagine ufficiale da Docker Hub (posso omettere registry e namespace)**
Nome completo: `docker.io/library/ubuntu:latest`
`docker.io` → Registry (Docker Hub, sottinteso se omissso)
`library` → Namespace predefinito per immagini ufficiali
`ubuntu` → Repository
`:latest` → Tag (impostato automaticamente se omissso)
Comando per scaricare: `docker pull ubuntu:latest`
- **Immagine personalizzata su Docker Hub (posso omettere registry)**
Nome completo: `docker.io/myusername/myapp:1.0`
`docker.io` → Registry
`myusername` → Namespace (utente)
`myapp` → Repository
`:1.0` → Tag
Comando per scaricare: `docker pull myusername/myapp:1.0`
- **Immagine da un registry privato (no abbreviazione)**
Nome completo: `my-registry.com/projectX/backend:2.3`
`my-registry.com` → Registry privato
`projectX` → Namespace (può essere un progetto o un'organizzazione)
`backend` → Repository
`:2.3` → Tag
Comando per scaricare: `docker pull myusername/myapp:1.0`

5. Il filesystem dei container in esecuzione

- Le immagini dei container sono salvati nel registry del daemon docker, cioè nel filesystem dell'host in una directory predefinita all'interno della cosiddetta docker area, la directory `/var/lib/docker/`.
- Questa docker area può anche essere una partizione separata, avente un filesystem specializzato (ad esempio btrfs) per rendere efficiente la memorizzazione delle immagini dei container e delle loro modifiche, e montata in `/var/lib/docker` (<https://docs.docker.com/storage/storagedriver/btrfs-driver/>).
- Quando viene creato, un container possiede un filesystem iniziale che è quello contenuto nella propria immagine. **I container che eseguono a partire da una stessa immagine condividono in sola lettura il filesystem dell'immagine nella docker area.**
- Durante l'esecuzione, il container può modificare il contenuto del proprio filesystem. Le modifiche vengono salvate in uno strato (layer), "al top" rispetto all'immagine originale del container, che mantiene SOLO le differenze rispetto all'originale.
- Questo layer differenziale viene salvato sul filesystem dell'host, nella docker area. Si parla di filesystem di tipo Copy-on-Write (CoW). Ciò è efficiente perché esisterà solo una istanza (in sola lettura) dell'immagine originale e poi ogni container avrà un proprio copia con le sole modifiche, risparmiando spazio disco.
- **I container che nascono da una stessa immagine condividono in sola lettura l'immagine originale** (cioè il filesystem originale)
- ma **ciascun container aggiunge un proprio livello in lettura e scrittura in cui mantiene le differenze rispetto all'originale.**
- **Nel momento della terminazione del container le modifiche vengono congelate.**
- Nel momento della **rimozione del container queste modifiche vengono perse.**



5.1. Struttura incrementale (a layers) delle immagini dei container

- **Docker Commit:** Se salvo immagini con **docker commit**, l'immagine è costituita da un **unico grande layer** che contiene, in forma piatta, tutto il **filesystem** del container.
- **Docker Build:** Se invece **creo l'immagine** di un container **mediante docker build**, ciascun command del Dockerfile, che modifica il filesystem dell'immagine, **crea un layer che memorizza solo le differenze** del filesystem rispetto ai layer sottostanti, secondo il modello Copy-on-Write (CoW).
- **Ciascun layer**
 - è salvato in una directory dentro la docker area `/var/lib/docker/`,
 - **non è modificabile** ma si può eliminare se non è usato da altri layer o immagini,
 - **è condivisibile tra più layers o immagini**,
 - **è identificato da un digest univoco**, calcolato con una funzione hash partendo dal contenuto del layer stesso. In teoria potrebbero esistere due layer con stesso digest ma è improbabile.
- Un layer non è una immagine completa, tranne se è stata costruita con **docker commit**.
- **Immagine di container:** è un insieme ordinato di layer, linkati dal più in alto al più in basso. Ogni layer contiene le differenze rispetto ai layer sottostanti.
- **Eliminare immagini o layer:** Non possiamo eliminare una data immagine o un layer se prima non eliminiamo tutte le immagini e layer che sono state costruite a partire da quella immagine o layer.
- **Il filesystem di un container in esecuzione** è perciò formato:
 - **dai layers della sua immagine** (quella usata da `docker run`) **che non sono modificabili**;
 - **da un layer aggiuntivo, modificabile, salvato anch'esso nella docker area**, in cui il driver di storage **scrive le modifiche al filesystem effettuate dal container in esecuzione**.

5.2. Layers di una immagine di container

- Per sapere l'identificatore univoco (il digest) di una immagine posso usare il comando:

docker inspect --format='{{.Id}}' NOMEIMMAGINE

che darà un output di questo tipo:

sha256:59ce995dfc35485bb5d3298e6298bd982ad478113ca1fa84656ca2364e03f1c5

- NB: Se due immagini hanno lo stesso identificatore allora sono la stessa immagine.

- Per sapere quali sono i layer di una immagine di container posso usare due comandi:

1) **docker history --no-trunc NOMEIMMAGINE** <-- troppo output

che fa vedere una riga per layer e anche [parte dei] comandi usati per creare quel layer.

Potrebbero comparire delle righe contenenti <missing> se mancano dei layer.

- Potrebbero essere layer intermedi che ho eliminato pulendo la build cache, No problem.

- Potrebbero essere layer intermedi di una build multi stage, No problem.

- Potrebbe essere un problema, se ho eliminato io a mano dei file nella docker area.

In questo ultimo caso il container potrebbe non partire.

2) **docker inspect --format='{{ .RootFS.Layers}}' NOMEIMMAGINE** <--vede id layer e altro

oppure, **per vedere solo gli identificatori (digest) dei layer** senza []

docker inspect --format='{{ .RootFS.Layers}}' NOMEIMMAGINE | sed 's/\[//g;s/\]//g'

ad esempio, il comando seguente fa vedere

docker inspect --format='{{ .RootFS.Layers}}' vic/ubuntu_with_nc_netstat | sed 's/\[//g;s/\]//g'

sha256:5498e8c22f6996f25ef193ee58617d5b37e2a96decf22e72de13c3b34e147591

sha256:02e1f81e89b9f537199924669ba1794ca49b2c72cd84d15eba335f54884bb104

5.3. Condivisione di layers tra immagini

- **Domanda: se due immagini di container sono costruite a partire da una stessa immagine base, quella immagine base viene duplicata nella docker area?**
 - Se **due immagini di container sono costruite con docker build**, partendo da una stessa immagine base (quella indicata dal command FROM nel Dockerfile), allora **l'immagine base non viene duplicata e ciascuna delle due immagini mantiene solo le differenze rispetto alla base**, mediante dei layers, costruiti incrementalmente, a seconda dei commands presenti nei due Dockerfile.
- **Domanda: se due immagini sono uguali allora hanno gli stessi layers?**
 - Se due immagini sono uguali (stesso digest) allora condividono gli stessi layer, **perciò i layers non sono duplicati**.
- **Domanda: se due Dockerfile hanno nome diverso ma sono uguali come contenuto, allora generano gli stessi layers e la stessa immagine o no?**
 - Se due Dockerfile hanno nomi diversi e stanno in directory diverse ma:
 - hanno lo stesso contenuto
 - le directory in cui si trovano sono uguali, hanno gli stessi files, anche quelli nascosti con gli stessi attributi di files (data di creazione, ultima modifica, timestamp vari, etc etc)
 - le variabili di ambiente in cui eseguo docker build sono uguali
 - In generale il contenuto del contesto è uguale
 - **allora le immagini saranno uguali** (stesso digest e stessi layer **riutilizzati**)
 - e "docker build" riutilizzerà gli stessi layers senza crearne una copia. I layer condivisi non vengono duplicati nel filesystem dell'host grazie alla cache e allo storage driver (es. overlay2).

5.4. Prove sperimentali (1/2)

- **Esercizi 1: uso di apt autoclean**

Usare questi due Dockerfile e verificare quale genera l'immagine più grande.

```
FROM ubuntu:latest

RUN apt-get update
RUN apt-get -y install curl net-tools wget && \
    apt-get -y install aptitude gcc
RUN apt-get -y remove gcc
RUN apt-get autoclean
ENTRYPOINT [ "/bin/bash" ]
```

```
FROM ubuntu:latest

RUN apt-get update
RUN apt-get -y install curl net-tools wget && \
    apt-get -y install aptitude gcc && \
    apt-get -y remove gcc && \
    apt-get autoclean
ENTRYPOINT [ "/bin/bash" ]
```

- Dopo avere fatto i due build, usare il comando `docker images` per vedere le dimensioni delle due immagini.
- Morale dell'esercizio. SE ESEGUO `apt autoclean` IN UN COMANDO "RUN" **separato**, allora **NON RIDUCO LA DIMENSIONE DELL'IMMAGINE** PERCHE' AVRO'
 - UN LAYER IN CUI HO MESSO I PACCHETTI ED I FILE .deb
 - ED UN LAYER SUPERIORE IN CUI "TOLGO" QUEI FILE .deb ma in realtà aggiungo info.

5.4. Prove sperimentali (2/2)

- **Esercizi 2: strati di immagine derivata**
- Scaricare l'immagine di ubuntu
- Listare i layers dell'immagine ubuntu
- Visualizzare il comando principale ENTRYPOINT e CMD con inspect
- Usare un Dockerfile minimo che crea immagine con script e diverso ENTRYPOINT
 - Listare i layers dell'immagine ubuntu
 - Eseguire e vedere che accade
 - Visualizzare il comando principale ENTRYPOINT e CMD con inspect
- Usare un Dockerfile minimo che crea immagine con script e diverso CMD
 - Listare i layers dell'immagine ubuntu
 - Eseguire e vedere che accade
 - Visualizzare il comando principale ENTRYPOINT e CMD con inspect
- Usare un Dockerfile minimo che crea immagine con script che usa argomenti e diversi ENTRYPOINT e CMD
 - Listare i layers dell'immagine ubuntu
 - Visualizzare il comando principale ENTRYPOINT e CMD con inspect

5.5. Eliminazione di layer e immagini

- Se due immagini di container sono costruite, con `docker build`, partono da una stessa immagine base, le due immagini non duplicano l'immagine base bensì mantengono solo le differenze rispetto alla base.
- Tra queste immagini che si appoggiamo su altre, potrebbero esserci anche alcune **immagini intermedie che sono state create durante il build** per funzionare come una sorta di **cache per build successivi**.
- Per vedere anche queste immagini intermedie, occorre aggiungere il flag `-a` al comando `images`
`docker images -a`
- Perciò non possiamo eliminare una data immagine se prima non eliminiamo tutte le immagini che si appoggiano, cioè che sono state costruite a partire da quella immagine base.
- Dei layer se ne occupa il daemon `docker`, che ci impedisce di eliminare immagini che sono usate da qualche container, in esecuzione o stoppato, o che sono una delle basi per altre immagini di container.

5.6. Immagini dangling

- Potrebbero esserci anche alcune **immagini intermedie che sono state create durante il build** per funzionare come una sorta di **cache per build successivi**.
- Quando queste immagini non sono più referenziate da nessuna immagine o container, ad esempio perché è stato cambiato un Dockerfile, tali immagini sono dette «dangling» (penzolanti) nel senso che non sono più usate da nessuno.
- Per vedere solo queste immagini dangling possiamo usare il comando:

```
docker images -f dangling=true
```

- Possiamo chiedere al daemon docker di selezionare e poi eliminare le immagini dangling, con una composizione di comandi che sfruttano una command substitution:

```
docker rmi $( docker images -q -f dangling=true )
```

oppure

```
docker image prune
```

5.7. Prove sperimentali

- **Esercizi 3: creare e vedere immagini dangling**
- partendo dall'immagine ubuntu,
- Creare un Dockerfile che installa pacchetti
 - fare build dell'immagine
 - listare i layers di quell'immagine
- Modificare il Dockerfile nei primi comandi installa pacchetti diversi
 - fare build dell'immagine
 - listare i layers di quell'immagine
- Verificare se esistono immagini dangling

6. Docker Area – filesystem sovrapposto

- **Docker Area:** Le immagini e i layers dei container sono salvati nel registry del daemon docker, cioè nel filesystem dell'host in una directory (normalmente di nome overlay2) all'interno della cosiddetta **docker area**, la directory `/var/lib/docker/`.
- **Partizione separata specializzata:** la docker area può anche essere una partizione separata, avente un filesystem specializzato per livelli (ad esempio Overlayfs, Aufs, Btrfs) per rendere efficiente la memorizzazione delle immagini dei container e delle loro modifiche, e montata in `/var/lib/docker` (<https://docs.docker.com/storage/storagedriver/btrfs-driver/>).
- **Storage Driver:** normalmente però la docker area non è una partizione separata, nel qual caso un driver di storage di docker adatta il filesystem esistente alle proprie esigenze di salvataggio dei layer, dei metadati e delle informazioni per collegare i layers e così formare le immagini.
- **Sovrapposizione di filesystem specializzato.** Lo storage driver si occupa di creare uno strato che opera come un filesystem per livelli ma che non si appoggia ai settori di disco formattato bensì ai file e directory del filesystem sottostante. Scopo di questo filesystem sovrapposto alla docker area, è rendere più facile ed efficiente salvare i layer, ed i loro metadati, e collegarli tra loro a formare le immagini. In particolare: a) dei link vengono usati per collegare i layer; b) delle codifiche hash sono usate per identificare il contenuto dei layer. Va notato che questa sovrastruttura espone una interfaccia da filesystem e si sovrappone al filesystem sottostante, che perciò viene mantenuto.
- **Il Driver di Storage:** stabilisce dove collocare le immagini, come creare i layer e come collegarli l'uno all'altro per formare le immagini, ci impedisce di eliminare immagini che sono usate da qualche container, in esecuzione o stoppato, o che sono una delle basi per altre immagini di container.
- Lo **storage driver predefinito** di docker è overlay2, che realizza OverlayFS, ma ne esistono altri. Perciò la directory con i layers è overlay2, ma può avere nome diverso se uso un altro driver.